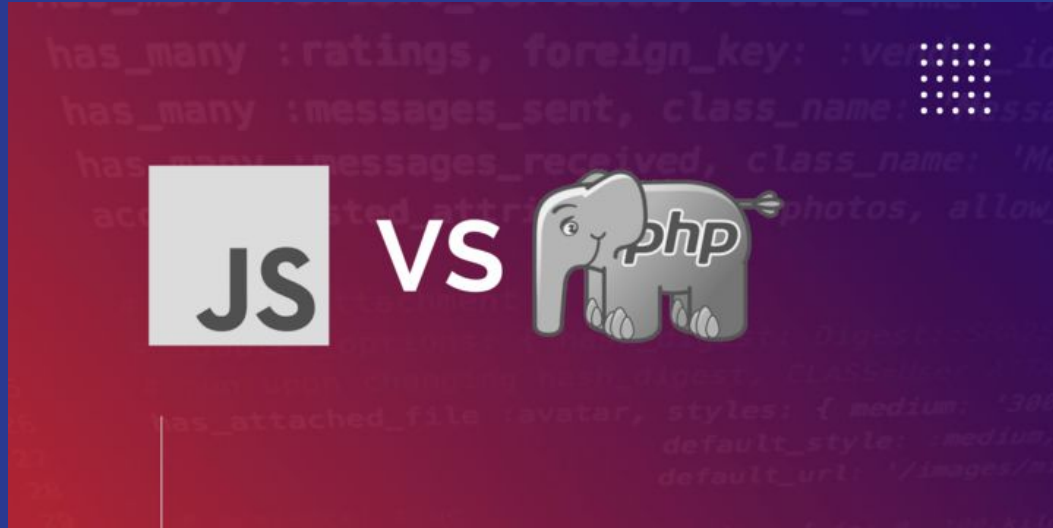


MÓDULO II

Configuração do Ambiente e Conceitos Básicos em PHP

MÓDULO II

PHP X JavaScript: Escolhendo a melhor linguagem para seus projetos



A Educação é o primeiro passo para um futuro melhor...

MÓDULO II

Se por acaso alguém disser que "**JavaScript** é apenas para desenvolvimento para **front-end** e **PHP** é para **back-end**", não dê ouvidos.

PHP é uma linguagem de programação voltada apenas para desenvolvimento para **back-end**. O **JavaScript**, por sua vez, foi inicialmente criado como uma linguagem para desenvolvimento para **front-end**. No entanto, com a introdução do **Node.js**, em **2009**, o **JavaScript** tornou-se o que chamamos de ***full-stack***.

Tendo isso em vista, a afirmação comumente citada de que "**JavaScript** é apenas para desenvolvimento para **front-end** e **PHP** é para **back-end**" simplesmente não é verdade. Hoje, você pode desenvolver toda uma aplicação usando JavaScript – tanto o lado do ***client*** quanto o lado do ***servidor***.
A questão é: qual linguagem é mais adequada para o seu projeto?

MÓDULO II

Portanto, se você está tentando decidir entre **PHP** e **JavaScript**, a discussão é reduzida a qual linguagem você deseja usar para o desenvolvimento do **back-end**. Para ajudá-lo a tomar essa decisão, compararei **PHP** e **JavaScript** para desenvolvimento para a web com base nos seguintes critérios:

1. *Visão geral*
2. *Desempenho e velocidade*
3. *Extensibilidade*
4. *Universalidade*
5. *Comunidade*
6. *Curva de aprendizagem*
7. *Sintaxe*
8. *Apps para os quais é mais adequado*

MÓDULO II



JavaScript



PHP

Brief Overview



A full stack language
Released: 1995

A back end language
Released: 1994

Performance & Speed



Faster

Slower

Extensibility



Can be combined with HTML, XML, and Ajax
A lot of available frameworks

Can be combined with HTML
Efficient CMS's

Universality



Cross-platform
Full-stack

Cross-platform
Back-end

Community



Large

Large

Learning



More difficult

Simpler

Syntax



Simple and compact

Simple and compact

Best suitable for



Any type, especially large projects

Blogs
Learning management systems
E-commerce websites

MÓDULO II

Visão geral

JavaScript

O **JavaScript** é uma linguagem de programação dinâmica, leve, multiparadigma, de alto nível, interpretada ou compilada *just-in-time*. Criado em **1995** por **Brendan Eich**, o **JavaScript** é caracterizado por sintaxe com uso de chaves (**{}**), funções de primeira classe e orientação a objetos baseada em protótipos.

De acordo com o site [Statista](#)(em inglês), **69%** dos desenvolvedores ao redor do mundo usam **JavaScript**. Outros **5%** pretendem adotar a linguagem. O relatório nos mostra que essa era a linguagem de programação mais popular no mundo ao no início desta década.

MÓDULO II

PHP

PHP é a sigla para ***PHP Hypertext Preprocessor***. É uma linguagem de script de código aberto para desenvolvimento para **back-end**. Desenvolvida em **1994** por **Rasmus Lerdorf**, a linguagem é mundialmente reconhecida. De acordo com o site W3Tech(em inglês), **79%** dos websites usam **PHP**. Entre os mais populares estão o *Facebook*, a *Wikipédia* e, claro, o *WordPress*.

PHP o x o JavaScript

MÓDULO II

Desempenho e velocidade

JavaScript

O **JavaScript** é caracterizado por um modelo de execução orientado por eventos, *sem bloqueio de entrada e saída e thread única*. Esse modelo é possível graças ao evento de *loop* e ao *clustering* do Node.

A natureza assíncrona do **Node.js** permite que ele percorra todo o código simultaneamente sem precisar esperar que algumas funções sejam executadas. Graças a isso, o **JavaScript** é a melhor solução para aplicações de baixa latência, como plataformas de *streaming*. O **Node.js** ganha ainda mais velocidade por conta do mecanismo V8, conexão constante com o servidor e funções de *callback*.

MÓDULO II

PHP

O **PHP** é caracterizado por um modelo de execução de *threads múltiplas com bloqueio de entrada e saída*. Ao contrário do **JavaScript**, o **PHP** é síncrono. A segunda linha de código em **PHP** só pode ser executada depois da primeira, o que o torna muito mais lento do que o **JavaScript**.

Embora o **PHP**, sem dúvida, perca em velocidade, alguns dizem que ele é mais estável do que o **JavaScript**. Porém, essa vantagem é duvidosa e ainda não foi totalmente comprovada.

*A confiabilidade e a estabilidade de ambas as linguagens é uma questão que permanece aberta à interpretação. Porém, graças à sua excelente velocidade, o **JavaScript** ganha um ponto no critério **desempenho e velocidade**.*

PHP 0 x 1 JavaScript

MÓDULO II

Extensibilidade

JavaScript

O JavaScript pode ser combinado com HTML, XML e Ajax.

Há um grande volume de *frameworks* para **JavaScript**. Seu número beira ao incontável, tendo em vista que novos surgem com bastante frequência.

As tecnologias de **JS** mais populares são: **Vue**, **Angular** e **React**.

O *framework* mais comum do lado do servidor é o **Node.js**. O *framework* que você escolher para desenvolver sua aplicação pode determinar sua velocidade e custo de desenvolvimento, assim como desempenho e outros fatores técnicos.

Com relação aos gerenciadores de pacotes, o **Node.js** já conta com o **NPM** (*Node Package Manager*) pré-instalado. O **NPM** facilita muito a vida dos desenvolvedores, sendo o maior registro de softwares do mundo.

MÓDULO II

PHP

O **PHP** pode ser combinado apenas com **HTML**.

Provavelmente, a maior vantagem do **PHP** é a disponibilidade de **CMSs**, como o **WordPress** ou o **Drupal**. Essas soluções podem facilitar muito o desenvolvimento para a web e até torná-lo mais barato. O **PHP** também pode ser estendido com qualquer tecnologia da **stack LAMP** e soluções de servidor, como **MySQL** ou **PostgreSQL**.

Existem dois gerenciadores de pacotes para o **PHP**: **PEAR** e **Composer**. O **PEAR** (*PHP Extension and Application Repository*) é uma biblioteca estruturada do **PHP** de código aberto. O **Composer** é uma ferramenta de gerenciamento de dependências para **PHP**.

*De maneira geral, o JavaScript oferece mais oportunidades de extensibilidade. Portanto, ele leva o ponto no critério **extensibilidade**.*

PHP o x 2 JavaScript

MÓDULO II

Universalidade

O **JavaScript** é multiplataforma, assim como o **PHP**. Tanto o **PHP** quanto o **JavaScript** são voltados, principalmente, para o desenvolvimento de aplicações para a web. Embora ambos possam ser utilizados para o desenvolvimento de aplicações móveis.

JavaScript

A maior vantagem do **JavaScript** sobre o **PHP** está no fato de o **JavaScript** ser uma linguagem de desenvolvimento ***full-stack***. A maior parte das comparações entre **JS** e **PHP** enfatizam que o **JavaScript** é uma linguagem usada apenas no **front-end**, o que não é verdade. Você pode desenvolver uma aplicação para a web ou para dispositivos móveis usando apenas o **JavaScript**.

Como uma empresa orientada para o **JS**, desenvolvemos soluções para a web e dispositivos móveis personalizadas a partir do zero usando apenas **JavaScript**.

MÓDULO II

PHP

O **PHP** é apenas uma linguagem de desenvolvimento para **back-end**.

Além disso, o **PHP** pertence à *stack* **LAMP**, que é uma sigla para: **Linux**, **Apache**, **MySQL** e **PHP/Perl/Python**.

Para desenvolver um aplicativo para a web com essa *stack* de tecnologias, um engenheiro de software precisa conhecer quatro sistemas de sintaxe diferentes — bem como **HTML** e **CSS**. Alternar entre linguagens é tão inconveniente quanto ineficiente, além de complicar muito a curva de aprendizagem.

*Desenvolver em **JavaScript** é desenvolver de maneira **full-stack**, sendo essa a maior vantagem da linguagem. Ponto para o **JavaScript** no critério **universalidade**.*

PHP 0 x 3 JavaScript

MÓDULO II

Comunidade

JavaScript

De acordo com o site stackshare.io, as principais razões que levam os desenvolvedores a gostar de trabalhar com o **JavaScript** são: sua **universalidade** (pode ser usado no **front-end** e no **back-end**), sua **popularidade** (está em todo lugar) e sua **extensibilidade** (conta com uma série de *frameworks* excelentes).

O **JavaScript** é usado pelas plataformas: **Netflix, LinkedIn, Trello, Uber, Airbnb, Instagram, eBay, NASA e Medium.**

A maioria dos *frameworks* de **JS** são de código aberto, embora o **JavaScript** propriamente dito não o seja.

O **JavaScript** é a linguagem mais popular do [GitHub](https://github.com), com mais de **20%** de *pull requests* compartilhados.

MÓDULO II

PHP

O stackshare.io nos mostra que as vantagens mais adoradas do **PHP** são: uma ampla comunidade, código aberto e facilidade de *deploy*.

O **PHP** é usado por empresas como **Facebook**, **Lyft**, **Wikipedia**, **Slack**, **Tumblr** e **9GAG**.

O **PHP** é uma linguagem de código aberto, o que o torna mais flexível e personalizável.

No **GitHub**, o **PHP** ocupa a oitava posição — com cerca de **5%** de *pull requests*.

*Ambas as linguagens têm enormes comunidades de suporte e são favorecidas por gigantes da tecnologia. Tanto o **PHP** quanto o **JavaScript** levam este ponto no critério **comunidade**.*

PHP 1 x 4 JavaScript

MÓDULO II

Curva de aprendizagem

PHP

PHP é muito mais simples de começar a aprender do que **JavaScript**. Configurar um servidor é tão simples quanto criar um arquivo **.php** — basta escrever algumas linhas de código envolvidas pelas tags **<?php?>** e colocar o **URL** no navegador.

Além disso, as particularidades do **PHP** — como funções inconsistentes ou valores de retorno — são mais fáceis de entender e de dominar do que as do **JavaScript** e de certos *frameworks* do **JS**.

MÓDULO II

JavaScript

Configurar o **JavaScript** — ou seja, o **Node.js** — para o lado do servidor não é tão complicado. É, porém, mais difícil do que seria com **PHP**. Um desenvolvedor de **JS** iniciante precisa de mais conhecimento sobre o assunto do que um desenvolvedor de **PHP** iniciante precisaria.

Porém, o esforço feito para se aprender **JavaScript** é totalmente recompensado. Por ser uma linguagem mais universal do que o **PHP**, o esforço de se aprender **JavaScript** acaba sendo mais valioso. *O **PHP** ganha este ponto pela sua simplicidade de aprendizagem. Fator que agrada aos desenvolvedores iniciantes.*

PHP 2 x 4 JavaScript

MÓDULO II

Sintaxe

A sintaxe de uma linguagem de programação, na maioria dos casos, é apenas uma questão de preferência pessoal. Portanto, nenhuma das linguagens ganha este ponto.

Ainda assim, apresentarei uma comparação entre as regras de sintaxe de ambas, para o caso de o leitor estar em dúvida sobre qual delas aprender. Talvez as particularidades de uma linguagem sejam mais adequadas para você. Talvez seja assim que você fará sua escolha.

- Tanto o **PHP** como o **JavaScript** fazem uso de parênteses, colchetes e chaves.
- As variáveis em **PHP** e em **JavaScript** podem ser de qualquer tipo, podem mudar de tipo e podem ter seu tipo verificado pelo respectivo operador de cada linguagem — *typeof* no **JS** e *gettype* no **PHP**.

MÓDULO II

- Em ambas as linguagens, os arrays iniciam em "0".
- O laço ***for()*** funciona de maneira idêntica no **PHP** e no **JavaScript**. A diferença está apenas em como a variável dentro dos parênteses é declarada. O laço ***foreach()*** do **PHP** pode ser facilmente convertido para um laço ***for()*** do **JavaScript**.
- Em **JS**, as variáveis têm escopo global por padrão, a não ser que declaremos seu escopo como local com a palavra reservada ***var***. Uma variável local está disponível para tudo dentro da função e de suas subfunções.
- Em **JS**, a adição e a concatenação são feitas com "+".

MÓDULO II

- O **JS** diferencia letras maiúsculas de minúsculas em variáveis e funções.
- Não há arrays associativos (pares de chave-valor) em **JS**.
É necessário usar strings do **JSON** para esse recurso.
- Em **JS**, arrays e objetos são muito semelhantes e, muitas vezes, intercambiáveis. Um item de objeto também pode ser referenciado como um item de array.
- Em **JS**, os itens de objetos são referenciados por meio do ponto final ".".
- O **PHP** faz uso de cifrões "\$" para indicar variáveis, coisa que o **JS** não faz. Todas as variáveis têm escopo local por padrão, a menos que declaradas globais com a palavra reservada *global*. Uma variável local não estará disponível em subfunções, a menos que a passemos como argumento.
- Em **PHP**, a adição é feita com " + " e a concatenação é feita com ".".

MÓDULO II

- O **PHP** diferencia letras maiúsculas de minúsculas APENAS em variáveis.
- O **PHP** permite arrays numéricos e associativos.
- Em **PHP**, arrays e objetos são duas coisas completamente diferentes — com sintaxes também diferentes.
- No **PHP**, itens em objetos são referenciados com uma seta "->".

Essas são algumas das diferenças mais básicas que você deve saber sobre as duas linguagens.

Como já foi mencionado, nenhuma das linguagens leva o ponto neste critério, pois a sintaxe é uma questão de preferência pessoal.

PHP 2 x 4 JavaScript

MÓDULO II

Aplicações para as quais é mais adequado

JavaScript

O **JavaScript** tem hospedagem de servidor dedicado, o que o torna perfeitamente adequado para grandes projetos. Ele pode ser usado para desenvolver para o **front-end** e para o **back-end** para quase qualquer tipo de aplicação de software — incluindo jogos **3D**, soluções **AR/VR**, produtos **IoT** e assim por diante.

MÓDULO II

PHP

Embora o **PHP** seja uma linguagem de programação de uso geral, ele é usado principalmente para o desenvolvimento de páginas web dinâmicas. Considerando a disponibilidade de sistemas de gerenciamento de conteúdo baseados em **PHP**, como **Moodle** e **WordPress**, o **PHP** é a melhor opção para blogs, sistemas de gerenciamento de aprendizagem e sites de *e-commerce*.

*Novamente, em termos de aplicações para as quais é mais adequada, cada linguagem é diferente, mas não necessariamente melhor ou pior do que a concorrente. Tanto o **PHP** quanto o **JavaScript** são bastante universais. Portanto, as duas linguagens levam este ponto.*

PHP 3 x 5 JavaScript

MÓDULO II

Para finalizar

A comparação entre **PHP** e **JavaScript** termina com um placar de **3 x 5** — conferindo a "**vitória**" para o **JavaScript**.

Ambas as linguagens são muito boas em termos de suporte da comunidade, extensibilidade e aplicações às quais são adequadas. O **JavaScript** é certamente mais eficiente em termos de velocidade e universalidade. Por outro lado, perde para o **PHP** em termos de curva de aprendizagem — embora a sintaxe, como concluímos, seja apenas uma questão de preferência pessoal.

Tenha em mente que nem sempre o **JavaScript** e o **Node.js** serão a melhor escolha. Isso cabe a você e ao seu projeto.

MÓDULO II

Vantagens e Desvantagens da Linguagem PHP

Vamos primeiro conhecer as principais vantagens da linguagem **PHP**.

Open Source e Grátis

Uma das maiores vantagens da linguagem **PHP** é ela ser de código aberto e gratuita.

É só baixar o serviço, integrar com um banco de dados e mandar ver no código para desenvolver para Web.
Não tem mistério.

MÓDULO II

O PHP é uma linguagem estável

Diferente das outras linguagens de programação para Web, o **PHP** é muito estável e pode fornecer assistência e suporte por muito tempo.

Essa característica foi conquistada por ser uma das linguagens de programação Web mais utilizadas no mundo.

É uma linguagem flexível.

O **PHP** é altamente flexível e os programadores podem mesclar suas funções com diversas outras linguagens e plataformas, como o **PYTHON**, **HTML**, **XML**, **PERL**, **RUBY**. Também é possível, integrar o **PHP** com banco de dados **MySQL**, **Oracle** e outros **SGBD's**.

MÓDULO II

Oportunidade de emprego

Caso você programe em **PHP**, provavelmente, haverá muitas vagas de emprego para você. Por ser uma linguagem de programação extremamente popular, o **php** possui uma vasta comunidade de desenvolvedores e empresas que utilizam a tecnologia.

PHP é multiplataforma

O **PHP** é uma linguagem que pode ser utilizada para diversas plataformas como **Windows, Linux, Mac** ... Isso é possível por ela ser uma linguagem interpretada e atuar do lado do servidor.

MÓDULO II

Desvantagens da Linguagem PHP

Chegou a vez de ver quais são as principais desvantagens do **PHP**, segue a lista:

Problemas de Segurança

Com o passar dos anos, a preocupação com os sistemas e sites desenvolvidos em **PHP** ainda persistem, por conta de vários motivos.

Um deles é por que a linguagem é **Open-Source** e as possíveis vulnerabilidades de código se tornam de conhecimento comum depois de serem exploradas. O código aberto permite que essas vulnerabilidades podem ser exploradas por qualquer programador com conhecimento e má intenção.

Outro motivo, é que por ser uma linguagem de fácil acesso, e por isso a porta de entrada para muitos desenvolvedores iniciantes na programação. Muitos deles não utilizam as devidas técnicas de segurança dentro do código o que causa vulnerabilidades e mancha a imagem da linguagem.

MÓDULO II

Popularidade em Decadência

Mesmo que, como vimos, o **PHP** seja uma poderosa linguagem WEB que possui uma vasta comunidade de desenvolvedores, profissionais e entusiastas, nos últimos anos surgiram linguagens mais fáceis para o desenvolvimento WEB, como o **Python**.

Atualmente o **php** domina o mercado Web, mas provavelmente o jogo virará no futuro e a linguagem ficará de lado.

MÓDULO II

Bibliotecas PHP x Novas Tecnologias

Nos últimos anos, diversas tecnologias como **Machine Learning**, **Data Science**, **Blockchain** tem surgido, e as bibliotecas e frameworks não acompanharam essa evolução. Provavelmente se sua empresa crescer baseada no **PHP**, ao precisar utilizar as tecnologias mais modernas, você não conseguirá.

Esqueça aplicações de grande porte

A performance fraca do **PHP**, disponibilidade de frameworks e libs, falhas de segurança, infelizmente, não vão permitir você ir muito longe. Se pretende escalar seu sistema para um nível mais alto, terá que lidar com muitos problemas se utilizar o **PHP**.

MÓDULO II

Depuração Limitada

Essa é talvez uma das maiores reclamações de quem programa em **PHP**: tem ferramenta de depuração muito limitadas. Os métodos de tratamento de erro são ruins, se comparado com outras linguagens de script, isso ocorre, por que as ferramentas de depuração necessárias para pesquisar e rastrear erros também não funcionam.

Conclusão

A linguagem de programação **PHP** ainda é bastante utilizada, justamente, por seus benefícios e simplicidade para programadores. É uma ótima linguagem de programação para o iniciante ao avançado... mas para níveis de aplicações mais robustas que necessite de performance e escalabilidade, pode não ser a linguagem de programação mais adequada.

MÓDULO II

Criando Scripts Dinâmicos: Interagindo com **HTML** e Incorporando Código **PHP**

O **PHP** é uma linguagem de programação amplamente utilizada para criar páginas web dinâmicas e interativas. Uma das suas principais características é a capacidade de incorporar código **PHP** diretamente em arquivos **HTML**, permitindo a criação de scripts dinâmicos. Vamos explorar como interagir com **HTML** e incorporar código **PHP** para criar páginas web dinâmicas.

MÓDULO II

Incorporando Código PHP em HTML

Para incorporar código **PHP** em um arquivo **HTML**, basta usar as tags especiais **<?php e ?>**. Todo o código **PHP** é colocado entre essas tags. Vamos dar uma olhada em um exemplo simples:

```
<!DOCTYPE html>
<html>
<head>
  <title>Página Dinâmica com PHP</title>
</head>
<body>

<h1>Olá, <?php echo "Mundo"; ?>!</h1>

<p>Hoje é <?php echo date("d/m/Y"); ?>.</p>

</body>
</html>
```

Neste exemplo, o **PHP** está sendo usado para inserir o nome “**Mundo**” no título **<h1>** e para exibir a data atual no parágrafo **<p>**.

MÓDULO II

Interagindo com HTML

Além de inserir dados dinâmicos em uma página **HTML**, o **PHP** também pode interagir com formulários **HTML** e processar os dados enviados pelos usuários. Vamos ver um exemplo de como criar um formulário de login simples e processar os dados com **PHP**:

```
<!DOCTYPE html>
<html>
<head>
  <title>Formulário de Login</title>
</head>
<body>

<h2>Formulário de Login</h2>

<form method="post" action="processar_login.php">
  <label for="usuario">Usuário:</label>
  <input type="text" id="usuario" name="usuario"><br><br>

  <label for="senha">Senha:</label>
  <input type="password" id="senha" name="senha"><br><br>

  <input type="submit" value="Entrar">
</form>

</body>
</html>
```

Neste exemplo, o formulário **HTML** envia os dados do usuário para um arquivo **PHP** chamado *“processar_login.php”* quando o botão *“Entrar”* é clicado.

MÓDULO II

Exemplo de Processamento de Formulário em PHP

Agora, vamos criar o arquivo “**processar_login.php**” para processar os dados do formulário:

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $usuario = $_POST["usuario"];
    $senha = $_POST["senha"];

    // Aqui você pode validar o usuário e a senha
    // e redirecionar o usuário para a página de destino

    if ($usuario == "admin" && $senha == "12345") {
        header("Location: pagina_restrita.php");
    } else {
        echo "Usuário ou senha inválidos!";
    }
}
?>
```

MÓDULO II

Neste exemplo, estamos recuperando os dados do formulário usando a variável superglobal **\$_POST** e realizando uma verificação simples. Se o usuário e a senha forem válidos, redirecionamos o usuário para uma página restrita. Caso contrário, exibimos uma mensagem de erro.

O **PHP** oferece uma maneira poderosa de criar páginas web dinâmicas e interativas. Incorporar código **PHP** em arquivos **HTML** permite que você crie scripts dinâmicos que podem interagir com o usuário, processar formulários e fornecer conteúdo personalizado. Experimente criar seus próprios scripts **PHP** para explorar ainda mais as possibilidades dessa linguagem de programação versátil e popular.

MÓDULO II

Manipulação de Dados: Usando Arrays e Loops para Processar Informações

No desenvolvimento web, muitas vezes é necessário manipular dados para criar páginas dinâmicas e interativas. O **PHP** oferece diversas ferramentas para lidar com isso, incluindo arrays e loops. Neste artigo, vamos explorar como usar arrays e loops para processar informações em **PHP**.

Arrays em PHP

Um array em **PHP** é uma coleção ordenada de elementos. Os elementos podem ser de diferentes tipos, como strings, números, booleanos ou até mesmo outros arrays.

MÓDULO II

Vamos ver como criar e manipular arrays em PHP:

```
// Criando um array
$cores = array("vermelho", "verde", "azul");

// Acessando elementos do array
echo $cores[0]; // Saída: vermelho

// Adicionando um novo elemento ao array
$cores[] = "amarelo";

// Iterando sobre os elementos do array
foreach ($cores as $cor) {
    echo $cor . "<br>";
}
```

MÓDULO II

Loops em PHP

Os loops são estruturas de controle que permitem executar um bloco de código várias vezes.

O **PHP** oferece várias formas de loops, incluindo **for**, **while**, **do-while** e **foreach**.

Vamos ver alguns exemplos:

Loop **for**:

```
for ($i = 0; $i < 5; $i++) {  
    echo "O valor de i é: " . $i . "<br>";  
}
```

MÓDULO II

Loop foreach:

```
$scores = array("vermelho", "verde", "azul");  
foreach ($scores as $score) {  
    echo $score . "<br>";  
}
```

Loop while:

```
$i = 0;  
while ($i < 5) {  
    echo "O valor de i é: " . $i . "<br>";  
    $i++;  
}
```

Loop do-while:

```
$i = 0;  
do {  
    echo "O valor de i é: " . $i . "<br>";  
    $i++;  
} while ($i < 5);
```


MÓDULO II

Exemplo de Manipulação de Dados em PHP

Vamos ver um exemplo prático de como usar **arrays** e **loops** para processar informações em **PHP**.
Suponha que temos um **array** de números e queremos calcular a soma de todos os elementos:

```
// Array de números
$numeros = array(1, 2, 3, 4, 5);

// Variável para armazenar a soma
$soma = 0;

// Loop para somar os números
foreach ($numeros as $numero) {
    $soma += $numero;
}

// Exibindo o resultado
echo "A soma dos números é: " . $soma;
```

MÓDULO II

Neste exemplo, usamos um loop **foreach** para iterar sobre os elementos do **array** e somá-los.
O resultado é então exibido na tela.

Arrays e **loops** são ferramentas poderosas para manipulação de dados em **PHP**.
Com **arrays**, você pode armazenar e organizar coleções de elementos, enquanto os **loops** permitem iterar sobre esses elementos e realizar operações em massa. Dominar o uso de **arrays** e **loops** abrirá muitas possibilidades para criar páginas web dinâmicas e interativas. Experimente criar seus próprios scripts **PHP** para explorar ainda mais esses conceitos e suas aplicações.

MÓDULO II

Como conectar ao banco de dados MySQL com PHP

Passo 1 - Criar o banco de dados:

Para criarmos um banco de dados, precisamos usar um gerenciador de banco de dados, **SGBD** (sistema de gerenciamento de banco de dados), em nosso artigo sobre o ambiente, instalamos o **PHPMyAdmin**, que é um gerenciador baseado em **PHP** para administrar o banco de dados **MySQL**. Este por sua vez, é largamente usado em quase **100%** dos servidores.

Além do **PHPMyAdmin**, podemos usar **SGBD** instalados em nosso computador, como o **MySQL Front**, **SQLyog**, **SQLAdmin**, **Sequel Pro**, entre outros.

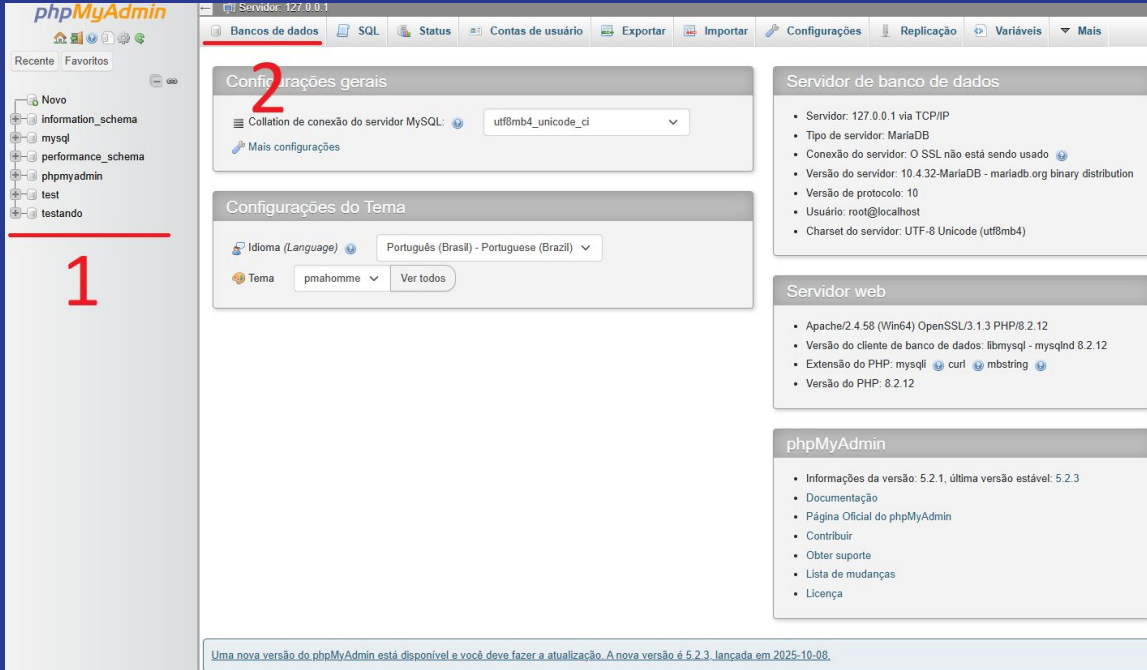
MÓDULO II

Abrindo o PHPMyAdmin:

- **WAMP:** localhost/phpmyadmin
- **MAMP:** localhost/MAMP/phpmyadmin
- **LAMP:** localhost/phpmyadmin
- **SERVIDOR ONLINE:** Consulte sua hospedagem

MÓDULO II

Depois de aberto, a tela inicial é esta:



- Lista dos bancos de dados já criados
- Aba onde mostra os bancos de dados já criados e possibilita a criação de novos.

Clique sobre o item 2.

MÓDULO II

Ao clicar sobre a aba Databases, você abrirá a tela abaixo

A Educação é o primeiro passo para um futuro melhor...

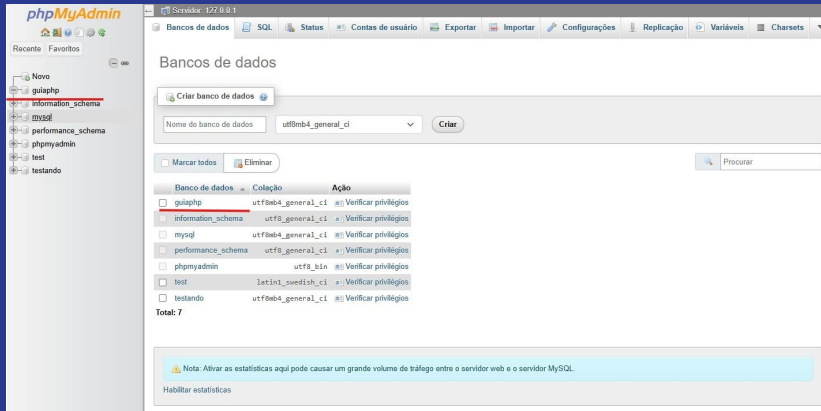
MÓDULO II

Esta tela mostra as bases já criadas, e deixar disponível uma opção para criar novos bancos de dados.

No campo destacado, você deve digitar o nome do banco de dados, ao lado no campo **Collation**, se refere ao tipo de linguagem e de dados que esta tabela vai conter, se você não escolher nada, como no exemplo acima, o sistema vai pegar o tipo padrão do banco de dados, o que é aconselhável. Depois de preenchido o nome, basta clicar no botão **Create**.

Imediatamente após criado o banco de dados, ele já estará disponível na barra lateral esquerda, conforme tela abaixo:

Agora sim, temos o banco de dados já criado, basta clicar sobre o nome dele para abrímos.



MÓDULO II

Passo 2 - Criando uma tabela no banco de dados criado:

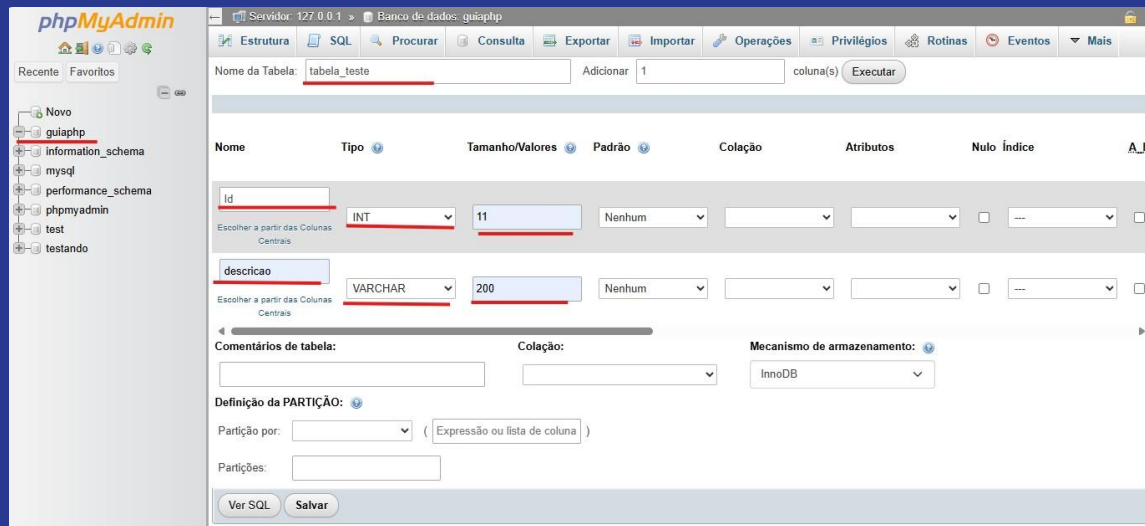
Depois de criar o **BD**, agora é necessário criar uma tabela neste banco de dados. A tabela é o local onde os registros ficarão armazenados. Cada **BD** pode ter uma ou mais tabelas. Para criar uma tabela, já estará o formulário na tela atual:



MÓDULO II

Basta preencher o nome da tabela, que no caso eu usei: **tabela_teste**, no segundo campo, precisamos preencher o número de colunas que esta tabela terá. Que serão os nomes dos campos da tabela. Neste caso coloquei 2, por que quero ter o campo chamado: ID e DESCRICAO. E então aperte o botão GO na direita.

Veja como foram criados os dois campos:



The screenshot shows the phpMyAdmin interface for a MySQL database named 'guiaphp'. The 'Estrutura' (Structure) tab is selected, and the 'tabela_teste' table is being created. The table has two columns:

Nome	Tipo	Tamanho/Valores	Padrão	Colação	Atributos	Nulo	Índice
id	INT	11	Nenhum			☐	☐
descricao	VARCHAR	200	Nenhum			☐	☐

Below the columns, the 'Comentários de tabela:' (Table Comments) field is empty. The 'Colação:' (Collation) field is set to 'utf8_general_ci'. The 'Mecanismo de armazenamento:' (Storage Engine) is set to 'InnoDB'. The 'Definição da PARTIÇÃO:' (Partition Definition) section is also empty. At the bottom, there are buttons for 'Ver SQL' and 'Salvar'.

MÓDULO II

ID é um campo de números, por isto o tipo (type) é **INT**. O tamanho do campo (Lenght) é **11**, ou seja, pode ter um número com até **11** algarismos, exemplo: **11111111111**.

Na imagem acima não aparece, mas existem um campo do tipo checkbox (check) com o nome de **A_I**, ou seja, **AUTO INCREMENT**. No campo **id** é preciso marcar ele, este **AUTO INCREMENT** funciona assim: quando um novo registro for inserido na tabela, o banco de dados automaticamente vai crescer este número, ou seja, se o último valor inserido for **14**, quando for acionado um novo **INSERT**, o próximo registro passa a ser **ID 15** e assim por diante.

O campo **DESCRIÇÃO**, colocamos do tipo **VARCHAR**, ou seja, **CHARACTER VARIÁVEL**, de tamanho **200**. Veremos mais adiante, os tipos de dados do banco, quando fizermos otimização de BD.

MÓDULO II

Bom, depois de inseridos os campos e configurados, basta apertar sobre o botão **SAVE**. Depois de salvo, a imagem abaixo irá aparecer, confirmando a inserção desta tabela e mostrando a tabela inserida:

The screenshot shows the phpMyAdmin interface. On the left is a sidebar with a tree view of databases: 'guiaphp' (selected), 'nova', 'tabela_teste', 'information_schema', 'mysql', 'performance_schema', and 'phpmyadmin'. The main area displays the 'Estrutura da tabela' (Table Structure) for 'tabela_teste' in the 'guiaphp' database. The table has two columns: 'id' (int(11), primary key, not null) and 'descricao' (varchar(255), utf8mb4_general_ci, not null). The interface includes various tabs at the top (Visualizar, Estrutura, SQL, etc.) and a bottom section with buttons for 'Imprimir', 'Propor estrutura de tabela', 'Monitorar tabela', 'Mover campo(s)', and 'Normalizar'. At the bottom, there is a form to add a new column, with '1' in the 'Adicionar' field and 'campo(s)' in the 'campo(s)' dropdown, and an 'Executar' button.

#	Nome	Tipo	Colaço	Atributos	Nulo	Padrão	Comentários	Extra	Ação
1	id	int(11)			Não	Nenhum		AUTO_INCREMENT	Alterar Eliminar Mais
2	descricao	varchar(255)	utf8mb4_general_ci		Não	Nenhum			Alterar Eliminar Mais

MÓDULO II

Passo 3 - Criar registros na tabela

phpMyAdmin

Servidor: 127.0.0.1 » Banco de dados: guaphp » Tabela: tabela_teste

Visualizar Estrutura SQL Procurar **Insérer** Exportar Importar Privilégios Operações Monitoramento Mais

Coluna	Tipo	Função	Nulo	Valor
Id	int(11)			
descricao	varchar(200)			registro 1

Executar

☒ Ignorar

Coluna	Tipo	Função	Nulo	Valor
Id	int(11)			
descricao	varchar(200)			registro 2 para teste

Executar

Certo, agora que já criamos o banco de dados e já criamos uma tabela, vamos inserir pelo menos dois registros na tabela para depois usá-los no código. Para isto basta clicar sobre a aba **INSERT** no **PHPMysqlAdmin**. Ao clicar, vai abrir uma tela com possibilidade de inserir dois registros. Você precisa preencher apenas a descrição destes dois registros, pois o campo **ID** é **AUTO INCREMENTO**.

MÓDULO II

Depois de inserir as descrições nos campos, aperte sobre o botão **GO** abaixo de ambos, que está destacado. Feito isto, terá uma tela de confirmação de registro. Agora aperte na aba **Browse**, nela veremos os dois registros devidamente inseridos, conforme tela abaixo:

Agora que temos o banco de dados, temos a tabela e já inserimos dois registros nela, a parte 1 de nosso tutorial está completa. Passamos agora para o código **PHP**.

Mostrando registros 0 - 1 (2 no total, Consulta levou 0.0003 segundos.)

```
SELECT * FROM `tabela_teste`
```

☒ Perfil [[Editar em linha](#)] [[Editar](#)] [[Demonstrar SQL](#)] [[Criar código PHP](#)] [[Atualizar](#)]

☒ Mostrar tudo | Número de linhas: Todos ▼ | Filtrar linhas: Procurar nesta tabela | Ordenar pela chave: Nenhum ▼

Opções extras

				id	descricao
☐	Editar	Copiar	Remover	1	registro 1
☐	Editar	Copiar	Remover	2	registro 2 para teste

↑ ☐ Marcar todos | Com marcados: Editar Copiar Remover Exportar

☒ Mostrar tudo | Número de linhas: Todos ▼ | Filtrar linhas: Procurar nesta tabela | Ordenar pela chave: Nenhum ▼

MÓDULO II

Passo 4 - Criando o arquivo de conexão:

No quarto passo vamos criar o arquivo de conexão ao **BD**, para isto usaremos duas funções nativas do **PHP**, chamadas de **mysql_connect()** e **mysql_select_db()**. Veja o código abaixo:

```
$servidor = 'localhost';  
$banco = 'guiaphp';  
$usuario = 'root';  
$senha = 'root';  
$link = mysql_connect($servidor, $usuario, $senha);  
$db = mysql_select_db($banco,$link);  
if(!$link)  
{  
echo "erro ao conectar ao banco de dados!";exit();  
}
```

MÓDULO II

As 4 primeiras linhas delimitamos os nomes das variáveis, qual servidor usaremos, qual o nome do banco de dados, o nome de usuário e a senha de usuário.

Depois usamos a função **mysql_connect()**, que tem como parâmetros: o servidor, usuário e senha. Uma vez conectado ao servidor, precisamos escolher qual banco de dados vamos trabalhar, aí que entra a função **mysql_select_db()**, cujos parâmetros são: nome do banco de dados e link da conexão do servidor, este link é a variável de saída da função **mysql_connect()**.

Com a variável \$link definida, podemos fazer um teste usando o **IF**, se der falso, a conexão com o servidor falhou.

MÓDULO II

Passo 5 - Selecionando dados da tabela

Depois de tudo que já fizemos, está na hora de pela primeira vez selecionar algum dado registrados nas tabelas. Para isto precisamos usar um comando chamado **SELECT** que é originário das linguagens **SQL**. Vou mostrar o comando mais simples possível, nos próximos itens do guia vamos desbravar a quantidade de códigos **SQL** existentes.

Para fazer o **SELECT**, precisamos usar a função **mysql_query()** do **PHP**, dando prosseguimento ao código acima usado, ou seja, precisamos estar conectados ao servidor e ao banco de dados que utilizaremos para somente então poder selecionar dados da tabela.

MÓDULO II

```
$servidor = 'localhost';
$banco = 'guiaphp';
$usuario = 'root';
$senha = 'root';
$link = mysql_connect($servidor, $usuario,
$senha);
$db = mysql_select_db($banco,$link);
if(!$link)
{
echo "erro ao conectar ao banco de dados!";exit();
}
```

```
$SQL = "SELECT * FROM tabela_teste";
$RS = mysql_query($SQL);
while($RF = mysql_fetch_array($RS))
{
echo '<pre>';
print_r($RF);
echo '</pre>';
}
```

MÓDULO II

```
/*  
RESULTADO:  
Array  
(  
[0] => 1  
[id] => 1  
[1] => registro 1  
[descricao] => registro 1  
)  
Array  
(  
[0] => 2  
[id] => 2  
[1] => registro 2 para teste  
[descricao] => registro 2 para teste  
)  
*/
```

Como pode ver, usamos o mesmo código para conectar ao banco de dados. Para fazer o **SELECT**, usamos a função **mysql_query()**, cujo parâmetro necessário é o comando SQL que vamos usar. Neste caso, usamos o comando: **SELECT * FROM tabela_teste**. Que se lê da seguinte maneira:

Selecione (**SELECT**) tudo (*) da (**FROM**) **tabela_teste**

MÓDULO II

Este comando vai retornar todos os registros e todos os campos da **tabela_teste**.

Depois disto, usamos a função **mysql_fetch_array()** em laço **WHILE**. Esta função precisa usar um laço pois ela retorna em um **ARRAY** cada uma das linhas, assim cada laço do **WHILE** é um item da tabela.

Como inserimos 2 registros na tabela, o **WHILE** fará obviamente 2 laços.

Dentro do **while**, usei um comando **print_r()** para mostrar o que o array **\$RF** recebe da função **mysql_fetch_array()**. Pode ver no resultado tudo que retornou no array.

MÓDULO II

```
'; print_r($RF); echo "; } /* RESULTADO: Array ( [0] => 1 [id] => 1 [1] => registro 1 [descricao] =>
registro 1 ) Array ( [0] => 2 [id] => 2 [1] => registro 2 para teste [descricao] => registro 2 para teste ) */ ?>
```

MÓDULO II

EXERCÍCIOS...

MÓDULO II

FIM...